

SIMATS ENGINEERING

CO2_LAB

#

```
=====
=====
# ARTIFICIAL NEURAL NETWORK USING BACKPROPAGATION
# Google Colab
#
=====
=====
```

```
import numpy as np
import matplotlib.pyplot as plt
```

#

```
=====
=====
# 1. CREATE DATASET - XOR
#
=====
=====
```

```
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)
```

```
y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)
```

```
print("Input Dataset:")
print(X)
```

```
print("\nTarget Output:")
print(y)
```

#

====

2. DEFINE ACTIVATION FUNCTIONS

#

====

def sigmoid(x):
return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
return x * (1 - x)

#

====

3. INITIALIZE NETWORK

#

====

np.random.seed(42)

Network structure:
Input Layer -> 2 neurons
Hidden Layer -> 4 neurons
Output Layer -> 1 neuron

input_neurons = 2
hidden_neurons = 4
output_neurons = 1

Initialize weights
W1 = np.random.uniform(
-1, 1,
(input_neurons, hidden_neurons)
)

W2 = np.random.uniform(
-1, 1,

```
(hidden_neurons, output_neurons)
)
```

```
    # Initialize biases
    b1 = np.zeros((1, hidden_neurons))
    b2 = np.zeros((1, output_neurons))
```

```
#
```

```
=====
=====
# 4. SET TRAINING PARAMETERS
```

```
#
```

```
=====
=====
    learning_rate = 0.5
    epochs = 10000
```

```
    loss_history = []
```

```
#
```

```
=====
=====
# 5. BACKPROPAGATION TRAINING
```

```
#
```

```
=====
=====
    for epoch in range(epochs):
```

```
    # -----
```

```
        # FORWARD PROPAGATION
```

```
    # -----
```

```
        # Hidden layer
        hidden_input = np.dot(X, W1) + b1
        hidden_output = sigmoid(hidden_input)
```

```
        # Output layer
        final_input = np.dot(
            hidden_output,
```

W2
) + b2

final_output = sigmoid(final_input)

```
# -----  
# CALCULATE ERROR  
# -----
```

error = y - final_output

loss = np.mean(error ** 2)

loss_history.append(loss)

```
# -----  
# BACKPROPAGATION  
# -----
```

Output layer gradient
output_delta = (
error *
sigmoid_derivative(final_output)
)

Hidden layer error
hidden_error = np.dot(
output_delta,
W2.T
)

Hidden layer gradient
hidden_delta = (
hidden_error *
sigmoid_derivative(hidden_output)
)

```
# -----  
# UPDATE WEIGHTS AND BIASES  
# -----
```

```

        W2 += (
            learning_rate *
            np.dot(
                hidden_output.T,
                output_delta
            )
        )

        b2 += (
            learning_rate *
            np.sum(
                output_delta,
                axis=0,
                keepdims=True
            )
        )

        W1 += (
            learning_rate *
            np.dot(
                X.T,
                hidden_delta
            )
        )

        b1 += (
            learning_rate *
            np.sum(
                hidden_delta,
                axis=0,
                keepdims=True
            )
        )

    # Display loss
    if epoch % 1000 == 0:
        print(
            f"Epoch {epoch}, Loss = {loss:.6f}"
        )

```

#

6. TEST THE TRAINED NETWORK

#

**hidden_output = sigmoid(
np.dot(X, W1) + b1
)**

**final_output = sigmoid(
np.dot(hidden_output, W2) + b2
)**

Convert probabilities into class labels

**predictions = (
final_output >= 0.5
) .astype(int)**

#

7. DISPLAY RESULTS

#

**print("\n")
print("=" * 60)
print("FINAL PREDICTIONS")
print("=" * 60)**

for i in range(len(X)):

**print(
f"Input: {X[i]} | "
f"Actual: {int(y[i][0])} | "
f"Predicted: {predictions[i][0]} | "
f"Probability: {final_output[i][0]:.4f}"**

)

#

=====

8. CALCULATE ACCURACY

#

=====

```
accuracy = np.mean(
    predictions == y
) * 100
```

```
print("\nAccuracy:", accuracy, "%")
```

#

=====

9. PLOT TRAINING LOSS

#

=====

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(
    loss_history
)
```

```
plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error")
```

```
plt.title(
    "Backpropagation Training Loss"
)
```

```
plt.grid(True)
```

```
plt.show()
```

#

10. TEST CUSTOM INPUT

#

print("\n")
print("=" * 60)
print("CUSTOM INPUT TEST")
print("=" * 60)

a = int(input("Enter first input (0 or 1): "))
b = int(input("Enter second input (0 or 1): "))

custom_input = np.array([
[a, b]
], dtype=float)

Forward propagation

custom_hidden = sigmoid(
np.dot(custom_input, W1) + b1
)

custom_output = sigmoid(
np.dot(custom_hidden, W2) + b2
)

custom_prediction = (
custom_output >= 0.5
).astype(int)

print("\nInput:", [a, b])

print(
"Output Probability:",
round(custom_output[0][0], 4)
)


```
print(
    "Predicted Class:",
    custom_prediction[0][0]
)
```

Input Dataset:

```
[[0. 0.]
 [0. 1.]
 [1. 0.]
 [1. 1.]
```

Target Output:

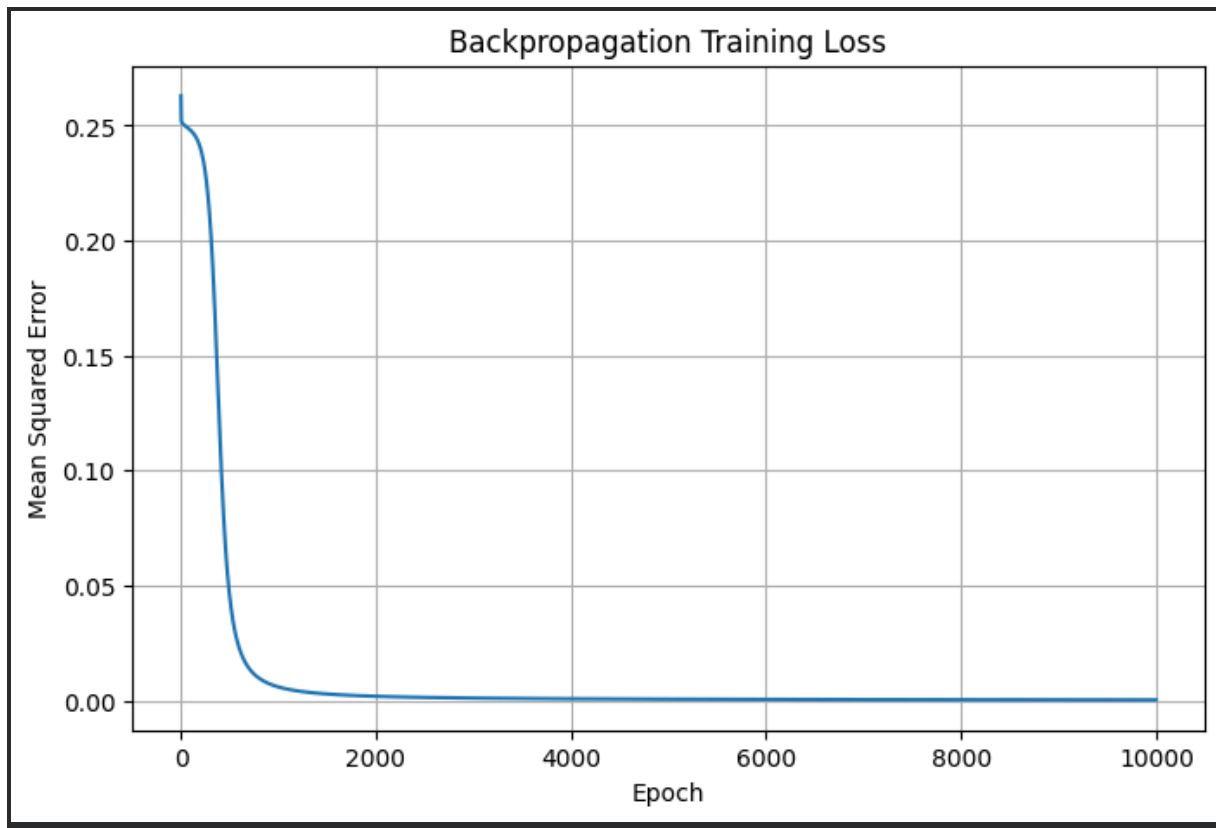
```
[[0.]
 [1.]
 [1.]
 [0.]
```

```
Epoch 0, Loss = 0.262916
Epoch 1000, Loss = 0.005853
Epoch 2000, Loss = 0.001787
Epoch 3000, Loss = 0.001019
Epoch 4000, Loss = 0.000705
Epoch 5000, Loss = 0.000536
Epoch 6000, Loss = 0.000431
Epoch 7000, Loss = 0.000360
Epoch 8000, Loss = 0.000309
Epoch 9000, Loss = 0.000270
```

```
=====
FINAL PREDICTIONS
=====
```

```
Input: [0. 0.] | Actual: 0 | Predicted: 0 | Probability: 0.0095
Input: [0. 1.] | Actual: 1 | Predicted: 1 | Probability: 0.9839
Input: [1. 0.] | Actual: 1 | Predicted: 1 | Probability: 0.9849
Input: [1. 1.] | Actual: 0 | Predicted: 0 | Probability: 0.0195
```

Accuracy: 100.0 %



=====

CUSTOM INPUT TEST

=====

Enter first input (0 or 1): 1
Enter second input (0 or 1): 0

Input: [1, 0]
Output Probability: 0.9849
Predicted Class: 1